

№2_Архитектура_вычислительных_систем

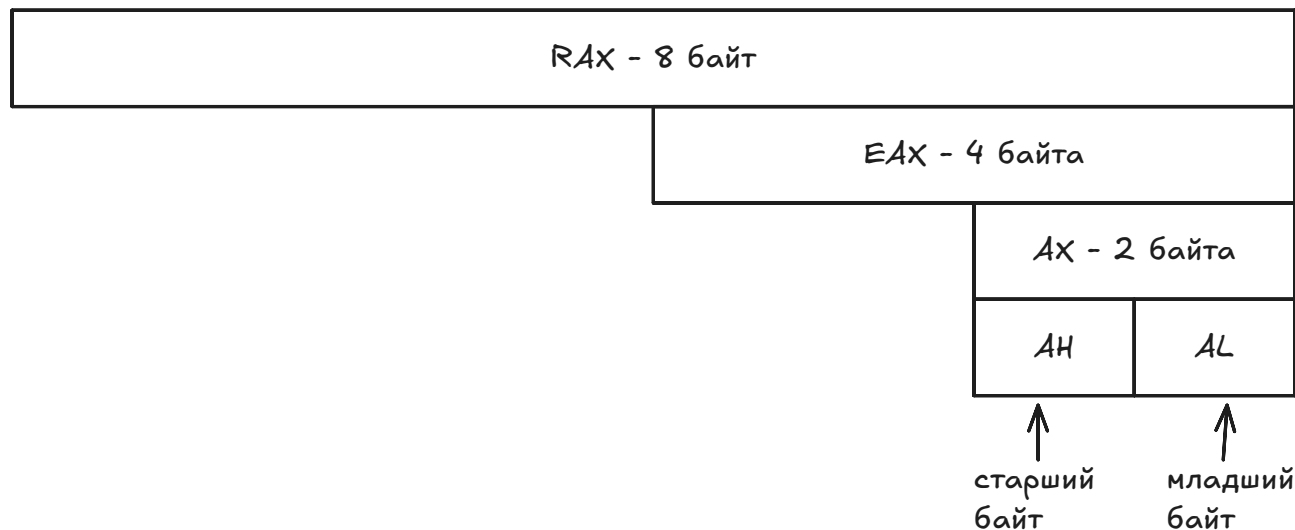
План

- Регистры процессора и структура программ
 - Объявление переменных в сегменте данных
 - Объявление и использование

Регистры процессора и структура программ

Регистры процессора - это ячейки быстродействующей памяти, расположенные непосредственно на кристалле процессора.

Структура 64-битного регистра *RAX*:



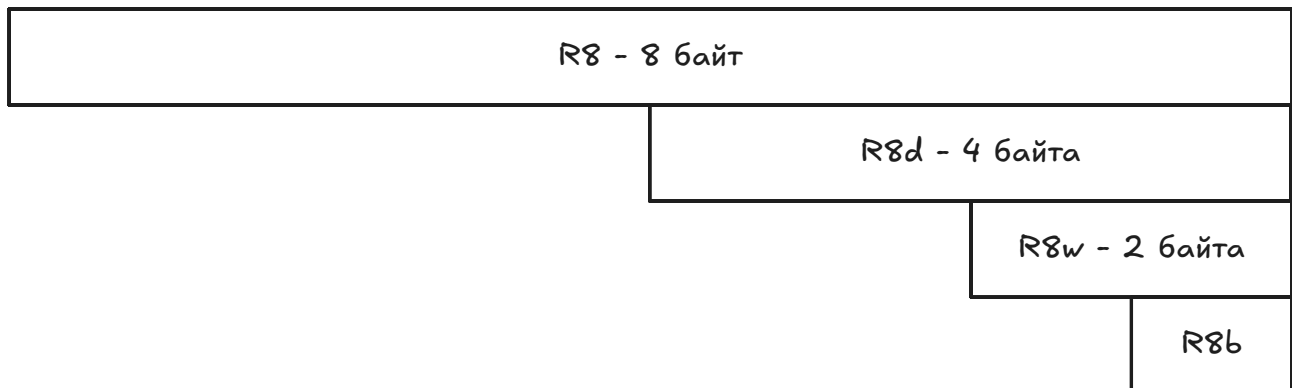
Виды общего назначения регистров:

- *EAX* - регистр *аккумулятор*.
- *EBX* - *базовый* регистр.
- *ECX* - регистр *счётчик*.
- *EDX* - регистр *данных*.
- *ESI* - *индексный* регистр *источника*.

- *EDI* - индексный регистр приёмника.
- *EBP* - регистр указатель на участок памяти (обычно в стеке).
- *ESP* - указатель на вершину стека.

Для 64-битных процессоров разработана архитектура *x64* в которую добавлены ещё восемь регистров общего назначения. *R8 – R15*.

Структура регистра *R8*:



Кроме регистров общего назначения существуют регистры:

- *EIP* - регистр указатель на текущую инструкцию. Он содержит адрес текущей инструкции в сегменте кода. Каждый раз при выполнении команды он увеличивается на размер текущей команды, за исключением команд перехода.
- *EFLAGS* - регистр, который содержит флаги процессора.

Флаг - это один бит, который содержит определённые результаты выполнения последней команды.

Объявление переменных в сегменте данных

Любая программа состоит из следующих сегментов:

1. *Сегмент данных* `.data`. В данном сегменте хранятся глобальные переменные.
2. *Сегмент кода* `.code`. В данном сегменте хранятся машинные коды команд программы.
3. *Сегмент стека*.

Директива	Размер	Си
db	1	char
dw	2	short
dd	4	int, float, long
df	6	
dq	8	long long? double
dt	10	

Объявление и использование

```
.data
a db 5                ; переменная
mas db 1, 2, 3        ; массив из трёх чисел
b dd 500000           ; большое число (0007A120h)
x dd -500000          ; отрицательное число (FFF85EE0h)

c db 4 dup(5)         ; эти две записи эквивалентны
c db 5,5,5,5,5

b dw 3 dup(1,2)       ; 1,2, 1,2, 1,2

s db "test", 0        ; строка (массив из ASCII кодов
символов)

fmt db "%d", 0        ; строка формата

.code
mov al, a             ; поместить 05 в al

push 7                ; поместить 07 в стек
push offset fmt       ; поместить fmt в стек
call crt_printf       ; вызвать функцию (аргументы в стеке)
```

Представление в сегменте данных:

```
05 01 02 03 20 A1 07 00 E0 5E F8 FF
```

Целые числа хранятся в оперативной памяти согласно двум правилам:

1. Числа хранятся в дополнительном коде.
2. Младший байт числа хранится по младшему адресу.

В ассемблере предусмотрена возможность повторять фрагменты в сегменте данных.